

The approach of Bertsimas and Periakakis to dynamic pricing consists of three steps.

- Demand estimation
- Estimation of the competitor's price
- (Myopic) own price setting

The demand model is

$$d_{k,t} = \beta_{k,t}^0 + \beta_{k,t}^1 p_{1,t} + \beta_{k,t}^2 p_{2,t} + \epsilon_{k,t}$$

where $k=1$ refers to you and $k=2$ refers to your competitor.

The input to the demand model is

- your own prices up to sales period $t-1$
- the quantity of demand you sold at your price level up to period $t-1$
- your competitor's prices up to sales period $t-1$

We want to minimize the deviation from the observed values to the estimated values given the demand model.

$$\min \sum_{\tau=1}^{t-1} \left| d_{1,\tau} - (\hat{\beta}_{1,\tau}^0 + \hat{\beta}_{1,\tau}^1 p_{1,\tau} + \hat{\beta}_{1,\tau}^2 p_{2,\tau}) \right|$$

such that $|\hat{\beta}_{1,\tau}^i - \hat{\beta}_{1,\tau+1}^i| \leq \delta_1(i)$ for $i=0, 1, 2$ and $\tau=1, \dots, t-2$

This nonlinear optimization problem can be linearized as follows:

$$\min \sum_{\tau=1}^{t-1} \Delta_{\tau}$$

such that $\Delta_{\tau} \geq |d_{1,\tau} - (\hat{\beta}_{1,\tau}^0 + \hat{\beta}_{1,\tau}^1 p_{1,\tau} + \hat{\beta}_{1,\tau}^2 p_{2,\tau})|$ for all $\tau=1, \dots, t-1$ and $|\hat{\beta}_{1,\tau}^i - \hat{\beta}_{1,\tau+1}^i| \leq \delta_1(i)$ for $i=0, 1, 2$ and $\tau=1, \dots, t-2$

The first constraint can be linearized by considering the positive and negative part of the absolute value:

$$\Delta_{\tau} \geq d_{1,\tau} - (\hat{\beta}_{1,\tau}^0 + \hat{\beta}_{1,\tau}^1 p_{1,\tau} + \hat{\beta}_{1,\tau}^2 p_{2,\tau})$$

and

$$\Delta_{\tau} \geq -d_{1,\tau} + (\hat{\beta}_{1,\tau}^0 + \hat{\beta}_{1,\tau}^1 p_{1,\tau} + \hat{\beta}_{1,\tau}^2 p_{2,\tau})$$

The same holds for the second constraint, i.e.

$$\hat{\beta}_{1,\tau}^i - \hat{\beta}_{1,\tau+1}^i \leq \delta_1(i)$$

and

```
import numpy as np

# the test data
p2 = np.array([52, 50, 43, 11, 14, 10, 90, 12, 45, 28, 33, 24, 24, 24, \
               36, 35, 36, 36, 35, 35, 34, 34, 35, 36, 37, \
                37, 36, 36, 37, 37, 36, 37, 37, 38, 37, 37, 38, 37, 38, \
                37, 77, 64, 40, 40, 40, 41, 41, 41, 41, 40, \
                 41, 36, 41, 41, 41, 41, 41, 41, 41, 41, 41, 41, 41, 42, 42, \
                 6, 34, 34, 34, 34, 34, 34, 34, 34, 34, 34, 34, \
                  34, 34, 34, 34, 34, 34, 35, 35, 35, 35, 76, 34, 34, 34, \
                 34, 1, 31, 31, 31, 41, 31, 31, 31, 31, 31])

p1 = np.array([56, 50, 48, 41, 9, 12, 8, 49, 10, 28, 28, 30, 22, 23, \
               24, 30, 33, 34, 35, 33, 34, 32, 33, 34, 35, \
                36, 37, 34, 36, 36, 37, 34, 36, 36, 37, 35, 36, 37, 35, \
                37, 37, 57, 60, 38, 39, 40, 40, 41, 41, 41, \
                 38, 40, 34, 38, 40, 40, 41, 41, 41, 41, 41, 41, 42, \
                 42, 4, 19, 27, 30, 32, 33, 34, 34, 34, 34, \
                  34, 34, 34, 34, 34, 34, 34, 95, 95, 95, 95, 95, 95, \
                 95, 95, 95, 95, 95, 95, 95, 95, 95, 95])

d = \
np.array([1,1,1,0,1,0,2,0,2,2,2,1,1,1,1,1,0,1,2,0,0,5,1,1,0,0,1,0,3,0,\
          1,3,2,1,2,0,2,2,0,0,1,2,0,0,0,0,2,1,0,1,\
           1,0,2,1,2,3,2,1,3,1,3,2,1,0,0,1,2,1,1,0,0,1,1,0,0,0,0,0,0,0,0,\
           0,0,0,0,0,0,0,0,0,0,0,0,0,0,0])

comp_has_capacity = \
np.array([True,True,True,True,True,True,True,True,True,True,True,\
         True,True,True,True,\
          True,True,True,True,True,True,True,True,True,True,True,\
          True,True,True,\
          True,True,True,True,True,True,True,True,True,True,True,\
          True,True,True,\
          True,True,True,True,True,True,True,True,True,True,True,\
          True,True,True,\
          True,True,True,True,True,True,True,True,True,True,True,\
          True,True,False,\
          False,False,False,False,False,False,False,False,False,Fals\
                                     e,False,False,\
                                   False]))
```

```

delta = 0.05

# the model
from pyomo.environ import *
# setup the corresponding IP model
model = ConcreteModel('Demand_Estim_1')

# the beta variables for k = 1,2 (in Python 0 and 1) and all t in T
MaxT = 40
T = RangeSet(1,MaxT,1)
K = RangeSet(1,1,1)
model.beta0 = Var(K,T, within = NonNegativeReals)
model.beta1 = Var(K,T, within = NonPositiveReals)
model.beta2 = Var(K,T, within = NonNegativeReals)
model.deltaTau = Var(T, within = NonNegativeReals)

def obj_rule(m):
    return sum(m.deltaTau[t] for t in T)
model.obj = Objective(rule=obj_rule)

def first_rule(m, t):
    return d[t-1] - (m.beta0[1,t] + m.beta1[1,t] * p1[t-1] +
m.beta2[1,t] * p2[t-1]) <= m.deltaTau[t]
model.first = Constraint(T, rule = first_rule)

def second_rule(m, t):
    return -d[t-1] + (m.beta0[1,t] + m.beta1[1,t] * p1[t-1] +
m.beta2[1,t] * p2[t-1]) <= m.deltaTau[t]
model.second = Constraint(T, rule = second_rule)

def third_rule0(m, t):
    if t < MaxT:
        return m.beta0[1,t] - m.beta0[1,t+1] <= delta
    else:
        return Constraint.Feasible
model.third0 = Constraint(T, rule = third_rule0)

def fourth_rule0(m, t):
    if t < MaxT:
        return -m.beta0[1,t] + m.beta0[1,t+1] <= delta
    else:
        return Constraint.Feasible
model.fourth0 = Constraint(T, rule = fourth_rule0)

def third_rule1(m, t):
    if t < MaxT:
        return m.beta1[1,t] - m.beta1[1,t+1] <= delta
    else:
        return Constraint.Feasible
model.third1 = Constraint(T, rule = third_rule1)

```

```

def fourth_rule1(m, t):
    if t < MaxT:
        return -m.beta1[1,t] + m.beta1[1,t+1] <= delta
    else:
        return Constraint.Feasible
model.fourth1 = Constraint(T, rule = fourth_rule1)

def third_rule2(m, t):
    if t < MaxT:
        return m.beta2[1,t] - m.beta2[1,t+1] <= delta
    else:
        return Constraint.Feasible
model.third2 = Constraint(T, rule = third_rule2)

def fourth_rule2(m, t):
    if t < MaxT:
        return -m.beta2[1,t] + m.beta2[1,t+1] <= delta
    else:
        return Constraint.Feasible
model.fourth2 = Constraint(T, rule = fourth_rule2)

# start solution peocess
opt = SolverFactory('glpk')

results = opt.solve(model)

# extract solution
beta10 = list(model.beta0.get_values().values())
beta11 = list(model.beta1.get_values().values())
beta12 = list(model.beta2.get_values().values())

# Next tasks
# 1.) Compute your next beta-values for time period MaxT + 1
# 2.) Copy the model from above and adapt it to the price optimization
# 3.) Compute your competitor's next beta-values for time period MaxT
+ 1
# 4.) Choose your next price in an optimum way

# first task: Compute your next beta-values for time period MaxT + 1
taking the average over the last N time periods
N = 10
beta10t = np.average(beta10[MaxT-N+1:MaxT])
beta11t = np.average(beta11[MaxT-N+1:MaxT])
beta12t = np.average(beta12[MaxT-N+1:MaxT])

print(beta10t,beta11t,beta12t)

1.9564831261101199 -0.06495318263126075 0.04537589171754918

```

second task: Copy the model from above and adapt it to the price optimization using non-linear optimization and IPOPT

setup the corresponding IP model
model = ConcreteModel('Demand_Estim_2')

the beta variables for k = 1,2 and all t in T

delta2 = 0.01

T = RangeSet(1,MaxT,1)

K = RangeSet(2,2,1)

model.beta0 = Var(K,T, within = NonNegativeReals)

model.beta1 = Var(K,T, within = NonNegativeReals)

model.beta2 = Var(K,T, within = NegativeReals)

def obj_rule(m):

 return sum((p2[t-1] + (m.beta0[2,t] + m.beta1[2,t] * p1[t-1]))/(2.0 * m.beta2[2,t]))**2 for t in T)

model.obj = Objective(rule=obj_rule)

solve model without constraint in order to obtain a feasible solution

def third_rule0(m, t):

 if t < MaxT:

 return (m.beta0[2,t] - m.beta0[2,t+1])**2 <= delta2

 else:

 return Constraint.Feasible

model.third0 = Constraint(T, rule = third_rule0)

#def fourth_rule0(m, t):

if t < MaxT:

return -m.beta0[2,t] + m.beta0[2,t+1] <= delta

else:

return Constraint.Feasible

#model.fourth0 = Constraint(T, rule = fourth_rule0)

def third_rule1(m, t):

 if t < MaxT:

 return (m.beta1[2,t] - m.beta1[2,t+1])**2 <= delta2

 else:

 return Constraint.Feasible

model.third1 = Constraint(T, rule = third_rule1)

#def fourth_rule1(m, t):

if t < MaxT:

return -m.beta1[2,t] + m.beta1[2,t+1] <= delta

else:

return Constraint.Feasible

#model.fourth1 = Constraint(T, rule = fourth_rule1)

def third_rule2(m, t):

```

    if t < MaxT:
        return (m.beta2[2,t] - m.beta2[2,t+1])**2 <= delta2
    else:
        return Constraint.Feasible
model.third2 = Constraint(T, rule = third_rule2)

#def fourth_rule2(m, t):
#    if t < MaxT:
#        return -m.beta2[2,t] + m.beta2[2,t+1] <= delta
#    else:
#        return Constraint.Feasible
#model.fourth2 = Constraint(T, rule = fourth_rule2)

# solver options for ipopt can be added here to a dictionary
solver_options = dict()

# start solution peocess
opt = SolverFactory('ipopt', sense = 'minimize', options =
solver_options )

result = opt.solve(model, tee=True)

# display model and its solution
print(result)

```

Ipopt 3.11.1:

```

*****
*****
This program contains Ipopt, a library for large-scale nonlinear
optimization.
Ipopt is released as open source code under the Eclipse Public
License (EPL).
    For more information visit http://projects.coin-or.org/Ipopt
*****
*****

```

NOTE: You are using Ipopt by default with the MUMPS linear solver.
 Other linear solvers might be more efficient (see Ipopt
 documentation).

This is Ipopt version 3.11.1, running with linear solver mumps.

Number of nonzeros in equality constraint Jacobian...	0
Number of nonzeros in inequality constraint Jacobian..	234
Number of nonzeros in Lagrangian Hessian.....	357

Error in an AMPL evaluation. Run with "halt_on_ampl_error yes" to see
 details.

Error evaluating objective gradient at user provided starting point.

No scaling factor for objective function computed!

Total number of variables.....:	120
variables with only lower bounds:	80
variables with lower and upper bounds:	0
variables with only upper bounds:	40
Total number of equality constraints.....:	0
Total number of inequality constraints.....:	117
inequality constraints with only lower bounds:	0
inequality constraints with lower and upper bounds:	0
inequality constraints with only upper bounds:	117

iter	objective	inf_pr	inf_du	lg(mu)	d	lg(rg)	alpha_du
alpha_pr	ls						
0	2.0642000e+004	0.00e+000	1.34e+005	-1.0	0.00e+000	-	
0.00e+000	0.00e+000	0					
1	1.7246429e+004	0.00e+000	1.31e+005	-1.0	1.46e-003	8.0	
1.00e+000	1.00e+000f	1					
2	1.2840286e+004	0.00e+000	4.16e+005	-1.0	1.15e-002	7.5	
1.00e+000	3.88e-001f	2					
3	1.2060276e+004	0.00e+000	5.92e+005	-1.0	7.76e-004	8.9	
1.00e+000	1.00e+000f	1					
4	1.1672809e+004	0.00e+000	7.14e+005	-1.0	3.68e-004	9.3	
1.00e+000	1.00e+000f	1					
5	5.9258387e+003	0.00e+000	1.17e+006	-1.0	2.79e-003	8.8	
1.00e+000	1.00e+000f	1					
6	4.8795038e+003	0.00e+000	1.23e+005	-1.0	4.09e-004	8.3	
1.00e+000	1.00e+000f	1					
7	2.6596410e+003	0.00e+000	1.03e+005	-1.0	1.40e-003	7.8	
1.00e+000	1.00e+000f	1					
8	2.1245356e+003	0.00e+000	1.25e+005	-1.0	6.51e-004	8.3	
1.00e+000	1.00e+000f	1					
9	5.3218764e+002	0.00e+000	1.10e+006	-1.0	4.76e-003	7.8	
1.00e+000	1.00e+000f	1					
iter	objective	inf_pr	inf_du	lg(mu)	d	lg(rg)	alpha_du
alpha_pr	ls						
10	1.3242069e+002	0.00e+000	2.89e+005	-1.0	1.31e-003	7.3	
1.00e+000	1.00e+000f	1					
11	3.7706344e+001	0.00e+000	4.34e+004	-1.0	2.05e-003	6.8	
1.00e+000	1.00e+000f	1					
12	1.7433513e+001	0.00e+000	3.63e+004	-1.0	8.10e-003	6.4	
1.00e+000	4.15e-001f	2					
13	5.8575728e-001	0.00e+000	5.81e+003	-1.0	1.99e-003	5.9	
1.00e+000	1.00e+000f	1					
14	4.2258869e-003	0.00e+000	4.81e+002	-1.0	2.52e-004	5.4	
1.00e+000	1.00e+000f	1					
15	4.2299563e-003	0.00e+000	2.27e+002	-1.0	1.31e-002	-	9.86e-
001	1.00e+000f	1					
16	1.5738943e-002	1.88e-001	2.36e+001	-1.0	9.01e-001	-	2.43e-

```

001 1.00e+000f 1
17 7.4102726e-002 6.19e-002 1.97e+001 -1.0 9.42e-001 - 6.28e-
001 7.85e-001f 1
18 1.6450807e-002 5.12e-003 1.94e+001 -1.0 1.95e+000 - 7.30e-
001 7.63e-001f 1
19 1.6914329e-002 3.23e-003 3.03e+001 -1.0 4.43e-001 -
1.00e+000 4.18e-001h 1
iter objective inf_pr inf_du lg(mu) ||d|| lg(rg) alpha_du
alpha_pr ls
20 4.3102326e-002 0.00e+000 9.11e+000 -1.0 3.73e-001 -
1.00e+000 1.00e+000h 1
21 3.1981996e-002 0.00e+000 2.43e-001 -1.0 6.70e-002 -
1.00e+000 1.00e+000h 1
22 1.4387034e-003 0.00e+000 7.03e-001 -1.7 2.63e-002 -
1.00e+000 1.00e+000f 1
23 1.3327402e-003 0.00e+000 1.46e-003 -1.7 2.65e-003 -
1.00e+000 1.00e+000h 1
24 1.5049761e-007 0.00e+000 4.53e-002 -3.8 6.88e-003 -
1.00e+000 1.00e+000f 1
25 7.5831445e-008 0.00e+000 3.45e-006 -3.8 1.18e-005 -
1.00e+000 1.00e+000h 1
26 1.1443342e-011 0.00e+000 2.54e-006 -5.7 5.21e-005 -
1.00e+000 1.00e+000h 1
27 2.1052862e-017 0.00e+000 4.99e-010 -8.6 6.63e-007 -
1.00e+000 1.00e+000h 1

```

Number of Iterations.....: 27

```

                                (scaled)                                (unscaled)
Objective.....: 2.1052861924617965e-017
2.1052861924617965e-017
Dual infeasibility.....: 4.9889125398477314e-010
4.9889125398477314e-010
Constraint violation.....: 0.0000000000000000e+000
0.0000000000000000e+000
Complementarity.....: 2.5066654838225076e-009
2.5066654838225076e-009
Overall NLP error.....: 2.5066654838225076e-009
2.5066654838225076e-009

```

```

Number of objective function evaluations      = 34
Number of objective gradient evaluations     = 28
Number of equality constraint evaluations     = 0
Number of inequality constraint evaluations   = 34
Number of equality constraint Jacobian evaluations = 0
Number of inequality constraint Jacobian evaluations = 28
Number of Lagrangian Hessian evaluations    = 27
Total CPU secs in IPOPT (w/o function evaluations) = 0.030
Total CPU secs in NLP function evaluations    = 0.003

```

EXIT: Optimal Solution Found.

Problem:

- Lower bound: -inf
- Upper bound: inf
- Number of objectives: 1
- Number of constraints: 117
- Number of variables: 120
- Sense: unknown

Solver:

- Status: ok
- Message: Ipopt 3.11.1\x3a Optimal Solution Found
- Termination condition: optimal
- Id: 0
- Error rc: 0
- Time: 0.11469507217407227

Solution:

- number of solutions: 0
- number of solutions displayed: 0

third task: compute the beta-values for your competitor for the next time period

extract solution

```
beta20 = list(model.beta0.get_values().values())
beta21 = list(model.beta1.get_values().values())
beta22 = list(model.beta2.get_values().values())
```

N = 10

```
beta20t = np.average(beta20[MaxT-N+1:MaxT])
beta21t = np.average(beta21[MaxT-N+1:MaxT])
beta22t = np.average(beta22[MaxT-N+1:MaxT])
```

```
print(beta20t,beta21t,beta22t)
```

```
3.0225016935730973 1.5098979801663974 -0.7664416085577366
```

fourth task: compute your next price

once again, a nonlinear optimization problem, but now only in one variable

```
from scipy.optimize import minimize_scalar
```

because we want to maximize, we minimize the negative function

```
f = lambda x: -x * ( beta10t + beta11t * x + beta12t * (beta20t +
beta21t * x)/(-2.0 * beta22t))
```

```
res = minimize_scalar(f, method='bounded', bounds = (0, 100))
```

```
print(res.x)
```

```
50.49820549661392
```